

Discussion Unit: RDDs part 2

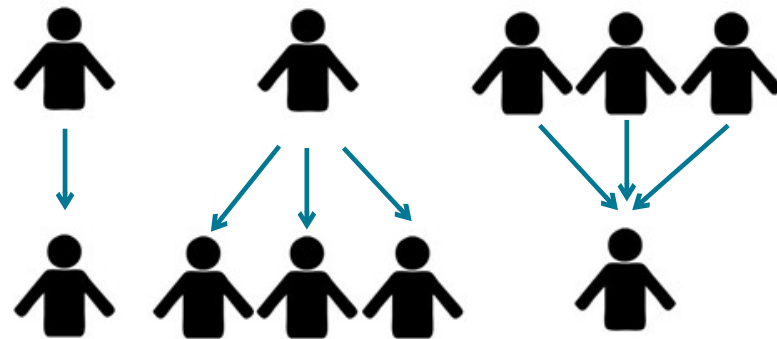
Complete the discussion
on RDDs

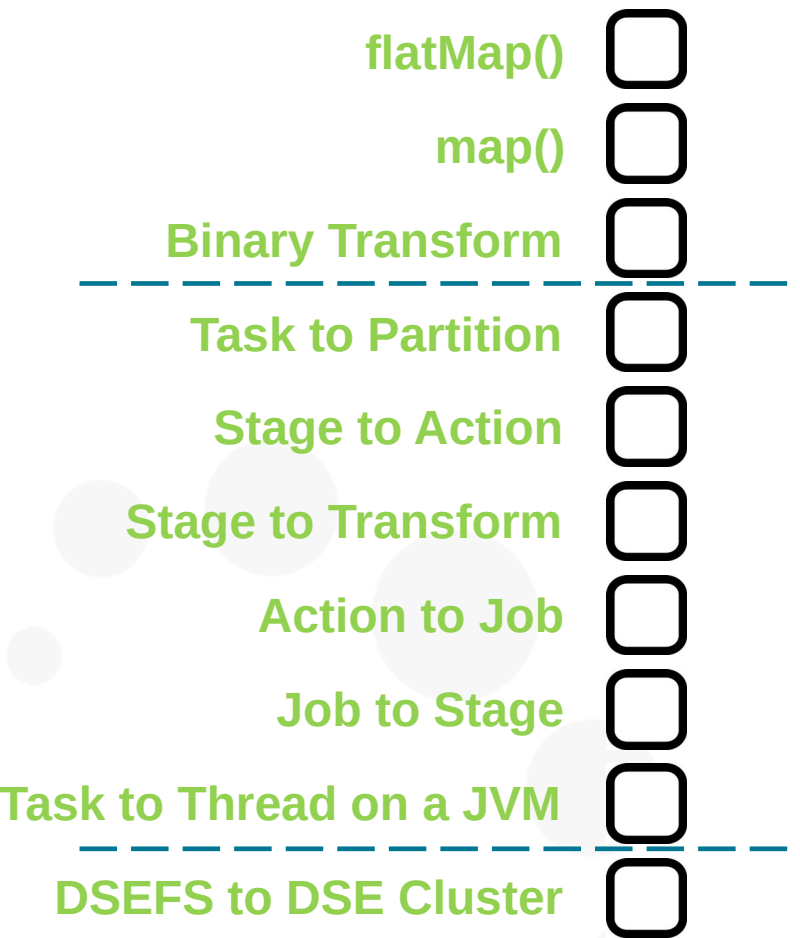
- Complete the topics relative to just RDDs
- More on `sc.CassandraTable()`, `sc.saveToCassandra()`
- More (Scala)/RDD examples, more transforms, more use cases

Discussion Lab:

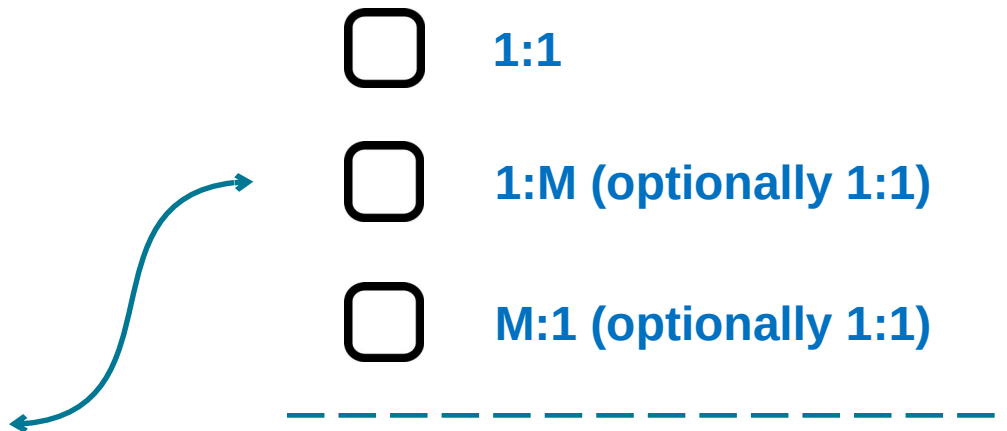
Matching pairs – Match the attributes on the right with the areas on the left

DSE Analytics; Cardinality between Terms, 1:1, 1:Many, Many:1





Relationship between Terms:



Logical, or Physical





End of Discussion Lab:

From: Maury_Atwater
To: DSE_HOTSHOT

Subject: Need this now !!!

Big deal. You previously loaded 3 tables. ☹

Now join the SQL modeled customer and orders, and load the DSE modeled customer_orders.

I also need Jobs for:

- . Customers without orders
- . Count of orders by state
- . Orders where paid_date is null
- . Count of orders by customer, sort by count descending

-MA

Working Goal:



Maury Atwater, President
of Atwater's

CQL Assets to Work with:

customer, orders, data



CQL Assets to Work With ..

```
DROP KEYSPACE IF EXISTS ks_7571;
```

```
CREATE KEYSPACE ks_7571 WITH REPLICATION =  
    {'class': 'SimpleStrategy',  
     'replication_factor': 1};
```

```
USE ks_7571;
```



CQL Assets to Work With ..

```
CREATE TABLE customer
```

```
(
```

```
customer_num
```

```
fname
```

```
lname
```

```
company
```

```
address1
```

```
address2
```

```
city
```

```
state
```

```
zipcode
```

```
phone
```

```
);
```

```
INT PRIMARY KEY,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT,
```

```
TEXT
```

// 6'th column ?

**OLD
OLD
OLD**

CQL Assets to Work With ..

```
INSERT INTO customer
(customer_num, fname, lname, company, address1,
address2, city, state, zipcode, phone)
VALUES ( 101, 'Ludwig', 'Pauli', 'All Sports Supplies',
'213 Erstwild Court', null, 'Sunnyvale', 'CA', '94086',
'408-789-8075');

VALUES ( 102, 'Carole', 'Sadler', 'Sports Spot',
'785 Geary St', null, 'San Francisco', 'CA', '94117',
'415-822-1289');

VALUES ( 103, 'Philip', 'Currie', 'Phils Sports',
'654 Poplar', 'P. O. Box 3498', 'Palo Alto', 'CA',
'94303', '415-328-4543');

VALUES ( 104, 'Anthony', 'Higgins', 'Play Ball!',
'East Shopping Cntr.', '422 Bay Road', 'Redwood City',
'CA', '94026', '415-368-1100');
```



Watch
address2
and nulls

```
CREATE TABLE customer2
(
  customer_num      INT,
  fname             TEXT,
  lname            TEXT,
  company           TEXT,
  address1          TEXT,
  address2         TEXT,
  city              TEXT,
  state             TEXT,
  zipcode           TEXT,
  phone            TEXT,
  PRIMARY KEY ((customer_num), company)
);
```

CQL Assets to Work With ..

**OLD
OLD
OLD**

Same data as customer,
different PK

```
CREATE TABLE orders
(
  order_num          INT PRIMARY KEY,
  order_date         TEXT,
  customer_num       INT,
  ship_instruct      TEXT,
  backlog            TEXT,
  po_num             TEXT,
  ship_date          TEXT,
  ship_weight        TEXT,
  ship_charge        TEXT,
  paid_date          TEXT
);
```

```
VALUES ( 1001, '05/20/1998', 104, 'express',
        'n', 'B77836', '06/01/1998', '20.4',
        '10.0', '07/22/1998');
```

(and 22 more orders)

CQL Assets to Work With ..



cassandraTable()

Examples using `cassandraTable()`



Scala (data): Tuples, Case Classes, other

```
val rows = sc.cassandraTable[ (Int, String) ]("ks_7545", "hello_world")  
// rdd.CassandraTableScanRDD[(Int, String)]
```

```
case class My_Record (pk: Int, value: String)  
//
```

```
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world")  
//rdd.CassandraTableScanRDD[My_Record]
```

sc.cassandraTable() .. From earlier



Pick one, don't need both

```
case class My_Record (pk: Int, value: String)
```

```
val rows = sc.cassandraTable[My_Record]("ks_7545", "hello_world").
```

```
  where("pk=111").
```

```
  select("pk", "value").
```

```
  as( (i:Int, s:String ) => new My_Record(i, s) )
```

```
rows.getClass()
```

```
// res: Class[_] = com.datastax.spark.connector.rdd.CassandraTableScanRDD
```

```
// [My_Record] = class com.datastax.spark.connector.rdd.CassandraTableScanRDD
```

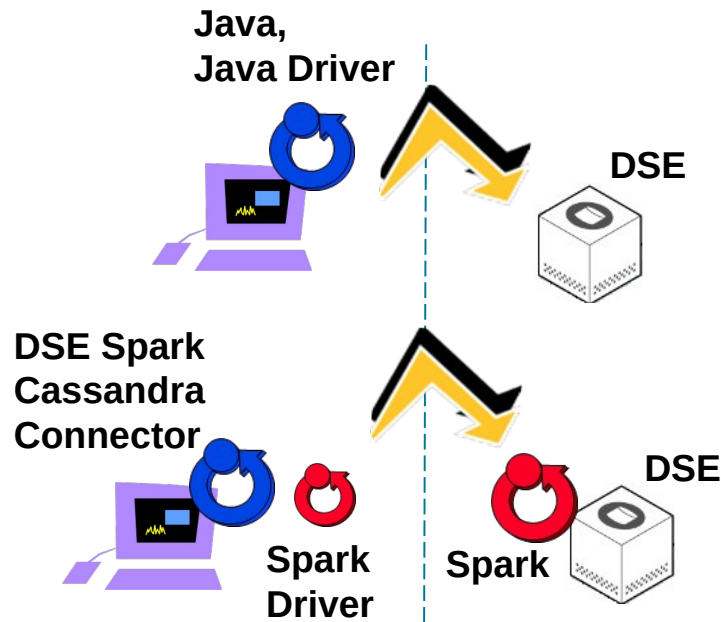
There are JDBC, ODBC, Parquet, (other) connectors built into Spark, but there is not a (Cassandra) one

DataStax provides 2 drivers that work interactively

- DSE Java Driver (See Discussion Unit, 6244/6245) (not Spark)
- **Spark Cassandra Connector (Spark)**
 - Open source, AND enterprise license features
 - Spark objects, activities

These allow you to connect to DSE directly from Spark, like using a JDBC driver, more

[What] is interacting with DSE ?



DSE Analytics Spark Cassandra Connector

API Call	Description
<code>cassandraTable(k, t)</code>	Returns RDD, SparkContext
<code>select(c1, c2), ..</code>	Optional, subset/all columns
<code>where(pred, [params], ..)</code>	Optional, WHERE clause, may be parameterized
<code>withAscOrder,</code> <code>withDescOrder</code>	Optional, ORDER BY, single DSE partition based on clustering columns, (runtime error if no CC)



DSE Analytics Spark Cassandra Connector

API Call	Description
limit(n)	Optional, how many rows to return
getType(c1) get[t](c1)	Return column of known Type/t, Scala equivalent of DSE column type; primitive, collection, UDT, tuple (runtime error if c1 does not exist, empty collections are returned)
getTypeOption(c1) get[O[t]](c1)	Best practice ; essentially, set a default value of (column) otherwise NULL
as(f)	Using function, map DSE cols to Scala; tuple, case class, other



DSE Analytics Spark Cassandra Connector

API Call	Description
<code>keyBy[k](c1, c2, ..)</code>	Convert DSE rows into Key/Value RDD, k and (cols) usually defined as Scala case class

Predicate Processing-



```
case class My_Record (pk: Int, value: String)
```

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  where("fname > 'D'")  
rows.collect().foreach(println)
```

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  filter(r => r.getString("fname") > "D")  
rows.collect().foreach(println)
```

Are these Jobs, Stages,
Tasks ?

How is each executed ?

Which has the great
likelihood to be more
efficient ?

If there is no (index or
similar), on the server
side, is one statement still
more efficient ?

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  where("fname > 'D' ")
```

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  where("fname > 'D' ").  
  withAscOrder
```

```
val rows = sc.cassandraTable("ks_7571", "customer2").  
  where("fname > 'D' ").  
  withAscOrder
```

```
val rows = sc.cassandraTable("ks_7571", "customer2").  
  withAscOrder
```

```
rows = sc.cassandraTable("ks_7571", "customer2").  
  where("customer_num = 101").  
  withAscOrder
```

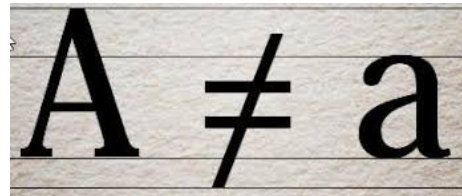
.withAscOrder()



Which compile ?
Which Work ?
When do they
fail ?

Case insensitive with a substring-

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  filter(r => r.getString("company").  
    toLowerCase.contains("sport"))  
rows.collect().foreach(println)
```



```
CassandraRow{customer_num: 103, address1: 654 Poplar, ...  
CassandraRow{customer_num: 102, address1: 785 Geary St, ...  
CassandraRow{customer_num: 101, address1: 213 Erswild Court, ...
```

Concatenate fields, default out null field



```
val rows = sc.cassandraTable("ks_7571", "customer").  
  map{ r => r.getString("address1") + ", " +  
    r.getStringOption("address2").getOrElse("N/A") +  
    ", " + r.getString("city") }  
rows.collect().foreach(println)
```

654 Poplar, P. O. Box 3498, Palo Alto
East Shopping Cntr., 422 Bay Road, Redwood City
785 Geary St, N/A, San Francisco
213 Erstwild Court, N/A, Sunnyvale

Handling nulls-

```
val rows = sc.cassandraTable[ (Int, String, String, String,  
    String, String, String, String,  
    String, String) ]("ks_7571", "customer")  
rows.collect().foreach(println)
```



```
// WARN 2018-07-20 17:37:40,994  
// org.apache.spark.scheduler.TaskSetManager:  
// Lost task 6.0 in stage 20.0 (TID 208, 127.0.0.1, executor 0):  
// java.lang.NullPointerException:  
// Unexpected null value of column address2 in ks_7571.customer.  
// If you want to receive null values from Cassandra, please wrap  
// the column type into Option or use JavaBeanColumnMapper
```

Handling nulls-



Why ?

```
val rows = sc.cassandraTable[ (Int, String, String, String,  
    String, Option[String], String, String,  
    String, String) ]("ks_7571", "customer")  
rows.collect().foreach(println)
```

// Still fails

```
val rows = sc.cassandraTable[ (String, String) ]("ks_7571", "customer").  
    select("address1", "address2")  
rows.collect().foreach(println)
```

// Still fails



Handling nulls-

```
val rows = sc.cassandraTable[ (String, Option[String]) ]("ks_7571", "customer").  
  select("address1", "address2")  
rows.collect().foreach(println)
```

```
(654 Poplar,Some(P. O. Box 3498))  
(East Shopping Cntr.,Some(422 Bay Road))  
(785 Geary St,None)  
(213 Erstwild Court,None)
```



Handling nulls-

```
case class My_Record ( address1: String, address2: Option[String])  
  //  
val rows = sc.cassandraTable[My_Record]("ks_7571", "customer").  
  select("address1", "address2")  
rows.collect().foreach(println)  
  
My_Record(654 Poplar,Some(P. O. Box 3498))  
My_Record(East Shopping Cntr.,Some(422 Bay Road))  
My_Record(785 Geary St,None)  
My_Record(213 Erstwild Court,None)
```



Handling nulls-

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("address1", "address2").  
  map(r => ( r.getString("address1"), r.getStringOption("address2") ))  
rows.collect().foreach(println)
```

```
(654 Poplar,Some(P. O. Box 3498))  
(East Shopping Cntr.,Some(422 Bay Road))  
(785 Geary St,None)  
(213 Erstwild Court,None)
```



Less efficient

Checking for nulls-

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("address1", "address2").  
  filter{ r => r.get[Option[String]]("address2").isDefined }  
rows.collect().foreach(println)
```

CassandraRow{address1: 654 Poplar, address2: P. O. Box 3498}
CassandraRow{address1: East Shopping Cntr., address2: 422 Bay Road}



Must use get()

Grouping and Sorting:



SQL Aggregation Example-

```
SELECT
  o.order_num,
  sum (i.total_price) price
FROM orders o, items i
WHERE o.order_date > '01/01/89'
  AND o.customer_num > 110
  AND o.order_num = i.order_num
GROUP BY 1
ORDER BY 2;
```



How many columns are output ?

As any aggregate query, (n types) of columns are output. What are these types ?

Which table is read first ?
What indexes would you have in place ?

Grouping, reduceByKey()

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("phone", "company").  
  map( r => ( r.getString("phone").substring(0,3), 1) ).  
  reduceByKey(_ + _)  
rows.collect().foreach(println)
```

(415,3)

(408,1)



What is the most precise name for the type of RDD that is output ?

What does "1" do ?

What is "_" ?

Grouping, groupByKey()

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("phone", "company").  
  map( r => ( r.getString("phone").substring(0,3),  
             r.getString("company") ) ).  
  groupByKey()  
rows.collect().foreach(println)
```

```
(415,CompactBuffer(Phils Sports, Play Ball!, Sports Spot))  
(408,CompactBuffer(All Sports Supplies))
```



How is this different
than the SQL query
that proceeded ?

Grouping, groupByKey and map()

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("phone", "company").  
  map( r => ( r.getString("phone").substring(0,3),  
    r.getString("company") ) ).  
  groupByKey().  
  map(r => (r._2.size, r._1, r._2))  
rows.collect().foreach(println)
```

```
(3,415, CompactBuffer(Phils Sports, Play Ball!, Sports Spot))  
(1,408, CompactBuffer(All Sports Supplies))
```



That's called a
Sequence



Grouping, groupByKey and map(), add sortBy()

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("phone", "company").  
  map( r => ( r.getString("phone").substring(0,3), r.getString("company") ) ).  
  groupByKey().  
  map(r => (r._2.size, r._1, r._2)).  
  sortBy(_. _1, true)  
rows.collect().foreach(println)
```

```
(1,408,CompactBuffer(All Sports Supplies))  
(3,415,CompactBuffer(Phils Sports, Play Ball!, Sports Spot))
```



What are the
arguments to
sortBy() ?

Grouping, countByKey()



```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("phone", "company").  
  map( r => ( r.getString("phone").substring(0,3), 1) ).  
  countByKey()
```

```
rows: scala.collection.Map[String,Long] = Map(415 -> 3, 408 -> 1)
```

A value is returned.

Why: Where is the
collect () ?

What type of RDD is
returned ?

Grouping: DSE side grouping

API Call	Description
<code>spanByKey()</code>	Group values for each key in the source Cassandra based RDD. Grouping is performed on the DSE side based on primary key columns.
<code>spanBy()</code>	Group elements of type V in the Cassandra based RDD. Grouping is performed on the DSE side based primary key columns.

Grouping: DSE side counting

API Call	Description
<code>cassandraCount()</code>	Returns a number of rows in the source Cassandra RDD.

Similar to where/filter: requires reference to whole of partitioning key

Joins: A Binary Transform



Joining-

```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state").  
  as((customer_num:Int, company:String, state:String) =>  
    (customer_num, (company, state)))  
  
val o_rows = sc.cassandraTable("ks_7571", "orders").  
  select("customer_num", "order_num", "order_date").  
  as((customer_num:Int, order_num:Int, order_date:String) =>  
    (customer_num, (order_num, order_date)))
```



What is up
with the green
parenthesis ?

What type of
RDD is this ?

Joining: join() (standard inner join)

```
val j_rows = c_rows.join(o_rows)
```

```
j_rows.collect().foreach(println)
```

```
(101,((All Sports Supplies,CA),(1002,05/21/1998)))  
(104,((Play Ball!,CA),(1013,06/22/1998)))  
(104,((Play Ball!,CA),(1011,06/18/1998)))  
(104,((Play Ball!,CA),(1001,05/20/1998)))  
(104,((Play Ball!,CA),(1003,05/22/1998)))
```



Also;
leftOuterJoin(),
rightOuterJoin(),
fullOuterJoin()

Joining: same, different (than prior page) ?

```
val j_rows = o_rows.join(c_rows)
```

```
j_rows.collect().foreach(println)
```

```
(101,((1002,05/21/1998),(All Sports Supplies,CA)))
```

```
(104,((1013,06/22/1998),(Play Ball!,CA)))
```

```
(104,((1011,06/18/1998),(Play Ball!,CA)))
```

```
(104,((1001,05/20/1998),(Play Ball!,CA)))
```

```
(104,((1003,05/22/1998),(Play Ball!,CA)))
```



How does the
output differ
than the prior
page ?

Joining: leftOuterJoin()

```
val j_rows = o_rows.leftOuterJoin(c_rows)
```

```
j_rows.collect().foreach(println)
```

```
(119,((1016,06/29/1998),None))
```

```
...
```

```
(115,((1010,06/17/1998),None))
```

```
(101,((1002,05/21/1998),Some((All Sports Supplies,CA))))
```

```
(116,((1005,05/24/1998),None))
```

```
...
```



Inner join ?
Outer join ?
How do they
differ ?

Joining: Where no match-pair exists

```
val j_rows = o_rows.leftOuterJoin(c_rows).  
  filter{case (pk, (row_o, row_c)) => !row_c.isDefined}
```

```
j_rows.collect().foreach(println)
```

```
(119,((1016,06/29/1998),None))
```

```
(112,((1006,05/30/1998),None))
```

```
(126,((1022,07/24/1998),None))
```

```
(120,((1017,07/09/1998),None))
```

```
...
```



Use case ?

Scala: How do you pick that apart ?

```
(119,((1016,null),Some(NJ)))  
(126,((1022,09/02/1998),Some(CO)))
```

```
map{r => ( r._1, r._2._1._2, r._2._1._1, r._2._2 )}
```



```
(119,null,1016,Some(NJ))  
(126,09/02/1998,1022,Some(CO))
```

```
map{r => ( r._1, Option(r._2._1._2), r._2._1._1, r._2._2 )}.  
map{r => ( r._1, r._2.getOrElse("N/A"), r._3, r._4.get )}
```

```
(119,N/A,1016,NJ)  
(126,09/02/1998,1022,CO)
```



The Some() will
come from an outer
join.

.get will ditch that

Tuple => Ordinals

Option(),
then .getOrElse to
lose the null

Joining: DSE side joining

```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state").  
  as((customer_num:Int, company:String, state:String) =>  
    (customer_num, (company, state)))  
  
val o_rows = sc.cassandraTable("ks_7571", "orders").  
  select("customer_num", "order_num", "order_date").  
  as((customer_num:Int, order_num:Int, order_date:String) =>  
    (customer_num, (order_num, order_date)))
```

Joining: DSE side joining

```
val j_rows = c_rows.joinWithCassandraTable("ks_7571", "orders").  
    on(SomeColumns("customer_num"))  
j_rows.collect().foreach(println)  
// java.lang.IllegalArgumentException: requirement failed:  
// Can't join without the full partition key. Missing: [ Set(order_num) ]
```

```
val j_rows = o_rows.joinWithCassandraTable("ks_7571", "customer").  
    on(SomeColumns("customer_num"))  
j_rows.collect().foreach(println)  
// java.lang.IllegalArgumentException: requirement failed:  
// Can't join without the full partition key. Missing: [ Set(order_num) ]
```

Joining: DSE side joining

```
val o_rows = sc.cassandraTable("ks_7571", "orders").  
  select("customer_num", "order_num", "order_date").  
  keyBy[Tuple1[Int]]("customer_num")  
val j_rows = o_rows.joinWithCassandraTable("ks_7571", "customer").  
  on(SomeColumns("customer_num"))  
  
j_rows.collect().foreach(println)
```

```
// ERROR 2018-07-21 20:46:03,465 org.apache.spark.scheduler.TaskSetManager:  
// Lost task 1.0 in stage 0.0 (TID 1, 127.0.0.1, executor 0):  
// com.datastax.spark.connector.types.TypeConversionException:  
// Cannot convert object (110) of type class scala.Tuple1 to java.lang.Integer.
```

Joining: DSE side joining

```
val o_rows = sc.cassandraTable("ks_7571", "orders").  
  select("customer_num", "order_num", "order_date").  
  keyBy(row => row.getInt("customer_num"))
```

```
val j_rows = o_rows.joinWithCassandraTable("ks_7571", "customer").  
  on(SomeColumns("customer_num"))
```

```
j_rows.collect().foreach(println)
```

```
((101,CassandraRow{customer_num: 101, order_num: 1002, ...  
((104,CassandraRow{customer_num: 104, order_num: 1013, ...  
...
```

Working with Key/Value RDDs



Key/Value RDD Unary Transforms

API Call	Description
mapValues(f)	Create new RDD, apply f() to each element from source RDD. <i>Keys retained without change.</i>
flatMapValues(f)	Same as above; one to many input to output, if f() returns a Sequence
reduceByKey(f, n)	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function func, which must be of type (V,V) => V. Like in groupByKey, the number of reduce tasks is configurable through an optional second argument.
foldByKey(z, n, f)	(Complex: See Notes page)



Key/Value RDD Unary Transforms

API Call	Description
combineByKey(..)	(Complex: See Notes page)
groupByKey(n)	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. ✓ te: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using reduceByKey or aggregateByKey will yield much better performance. ✓ te: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional numTasks argument to set a different number of tasks.
sortByKey(ASC, n)	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument.



Key/Value RDD Actions

API Call	Description
lookup(k)	(Example on pages that follow.)
collectAsMap()	Returns a map of key/value pairs from source RDD.
countByKey()	Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.



```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state").  
  as((customer_num:Int, company:String, state:String) =>  
    (customer_num, (company, state)))
```

```
val c_rows_k = c_rows.keys  
c_rows_k.collect().foreach(println)  
103  
104  
...
```

```
val c_rows_v = c_rows.values  
c_rows_v.collect().foreach(println)  
(Phils Sports,CA)  
(Play Ball!,CA)  
...
```

Key/Value RDD: (support) transforms



```
val rows2 = c_rows.  
  mapValues{case (v) => (v, "New Value")}  
rows2.collect().foreach(println)
```

```
(103,((Phils Sports,CA),New Value))  
(104,((Play Ball!,CA),New Value))  
...
```

```
rows2.getClass  
res: Class[_ <: org.apache.spark.rdd.RDD[(Int, ((String, String), String)]] = class  
org.apache.spark.rdd.MapPartitionsRDD
```

```
val map1 = rows2.collectAsMap()  
map1.getClass  
res: Class[_ <: scala.collection.Map[Int,((String, String), String)]] = class  
scala.collection.mutable.HashMap
```

Key/Value RDD:
[mapValues\(\)](#),
[collectAsMap\(\)](#)

Key/Value RDD: lookup()



What is the source
of that error ?

```
val row_102 = rows2.lookup(103)
row_102.collect().foreach(println)
// Not an RDD anymore
// error: not enough arguments for method collect:
// (pf: PartialFunction[((String, String), String), B])
// (implicit bf: scala.collection.generic.CanBuildFrom[Seq[((String, String), String)], B, That])
// That. Unspecified value parameter pf.
```

row_102

```
res: Seq[((String, String), String)] = WrappedArray(((Phils Sports,CA),New
Value))
```

**Key/Value RDD,
Binary
Transforms:
union(),
intersection(),
difference()**



"Union compatible", same number
and type of columns

Key/Value RDD: union(), intersection(), difference()

```
val c_rows1 = sc.cassandraTable("ks_7571", "customer").  
  where("customer_num = 101").  
  select("customer_num", "company", "state").  
  as((customer_num:Int, company:String, state:String) =>  
    (customer_num, (company, state)))
```

```
c_rows1.collect().foreach(println)
```

```
(101,(All Sports Supplies,CA))
```



What are the
key cols ?
Value cols ?

```
val c_rows2 = sc.cassandraTable("ks_7571", "customer").  
  where("customer_num > 102").  
  select("customer_num", "company", "state").  
  as((customer_num: Int, company: String, state: String) =>  
    (customer_num, (company, state)))  
  
c_rows2.collect().foreach(println)
```

```
// java.lang.UnsupportedOperationException:  
// Range predicates on partition key columns  
// (here: customer_num) are not supported in  
// where. Use filter instead.
```

Key/Value
RDD: union(),
intersection(),
difference()



Why does this
fail ?
When does
this fail ?

Key/Value
RDD: union(),
intersection(),
difference()

```
val c_rows2 = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state").  
  as((customer_num: Int, company: String, state: String) =>  
    (customer_num, (company, state))).  
  filter{case (customer_num, (company, state)) => (customer_num > 102)}  
  
c_rows2.collect().foreach(println)  
  
(103,(Phils Sports,CA))  
(104,(Play Ball!,CA))
```

Key/Value RDD: union()

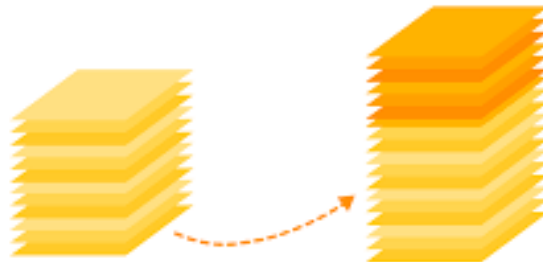
```
val u_rows = c_rows1.union(c_rows2)
```

```
u_rows.collect().foreach(println)
```

```
(101,(All Sports Supplies,CA))
```

```
(103,(Phils Sports,CA))
```

```
(104,(Play Ball!,CA))
```



Partitioning:



Partitioning:



- An (RDD) is a distributed collection of partitions
- Spark automatically partitions based on source and target, and type of action
- Rarely manually re-partition when using DataFrames, Datasets; no need, optimizer takes care of that.
- RDDs, no optimizer.
- But, RDDs will automatically match partitioning of DSE table due to DSE Analytics Spark Cassandra Connector
- *Reshuffling/repartitioning; perhaps the most expensive operation*

Partitioning: Rules, and more

- `RDD.partitions` Returns array of partition objects
- `RDD.partitions.size` Length of above
- `RDD.partitioner` Returns partitioner name
- Range partitioner, used for sorting
- None (partitioner), random
- Custom (partitioner), example: (DSE Core)
- Number of partitions == number of tasks (per stage)
- `sc.defaultParallelism`
 - Outputs setting
 - Default number of tasks that can execute concurrently
 - Equal to number of DSE cores, by default



Partitioning: Rules, and more



Default partitioning behavior-

Local Scala collection

None partitioner, `sc.defaultParallelism`

DSE Core table

None partitioner or,
`com.datastax.spark.connector.rdd. \`
`partitioner.CassandraPartitioner` (you decide)

`sc.defaultParallelism` or

`approx data size / 64MB`, whichever is greater

HDFS/DSEFS

None partitioner, `sc.defaultParallelism`, or
number of file blocks, whichever is greater

Partitioning: Rules, and more

Result of transforms-

- filter(), map(), flatMap(), distinct(), ..
- None partitioner, exception: filter uses parent

union(), Sum of two RDD partition sizes

intersect(), Max of two RDD partition sizes

subtract(), Parent RDD partitioner size

cartesian(), Two RDD partition sizes multiplied

Key based transforms-

- reduceByKey(), foldByKey(), combineByKey(), groupByKey, .. HashPartitioner, same as parent RDD
- mapValues(), flatMapValues(), Same as parent RDD
- cogroup(), join(), leftOuterJoin(), .. HashPartitioner, Same as parent RDD, or configurable

Partitioning: Setting-

- Optimal ? 2x number of cores
- 100ms+ Task execution time
 - Too few: possible data skew, increased memory pressure, longer recovery from failure
 - Too many: Task scheduling may take longer than task execution, more lineage information to record
- The custom partitioner used to read/write to/from DSE Core
- `val my_data = sc.parallelize(List(...)).
partitionBy(new org.apache.spark.HashPartitioner(9)) // 9 partitions`

Repartitioning/Shuffling-

Cost Factors

- Disk I/O
- Network traffic
- Partitioning execution
- External sorting
- Serialization/deserialization
- Data compression



Shuffling tunables: spark-env.sh

Property	Description	Default
spark.shuffle.manager	Data shuffling strategy; sort or hash	sort
spark.shuffle.spill	Enables spilling data to disk by reduce tasks	true
spark.shuffle.memoryFraction	Spilling threshold as fraction of Java heap	0.2
spark.shuffle.sort. bypassMergeThreshold	No merge-sort if number of reduce tasks not greater and no map-side aggregation	200
spark.shuffle.compress	Enable compression for shuffle writes	true
spark.shuffle.spill.compress	Enable compression for spilled data	true

Partitioning: Examples, suboptimal (from DSA)

```
val rows = sc.cassandraTable("k1", "t1").  
  keyBy(row => row.getInt("release_year")).  
  repartition(2*sc.defaultParallelism)
```

```
val rowsCountByYear = rows.countByKey().  
  foreach(println)
```

```
val rowsByYear = rows.groupByKey().  
  collect.  
  foreach(println)
```



3 shuffles;
repartition(),
countByKey(),
groupByKey()



Partitioning: Examples, optimal (from DSA)

```
val rows= sc.cassandraTable("k1", "t1").  
  keyBy(row => row.getInt("release_year")).  
  partitionBy(new org.apache.spark.HashPartitioner(2*sc.defaultParallelism)).  
  cache
```

```
val rowsCountByYear = rows.countByKey().  
  foreach(println)
```

```
val rowsByYear = rows.groupByKey().  
  collect.  
  foreach(println)
```



1 shuffle



Partitioning: Which operations cause reshuffling

- repartition
- coalesce
- partitionBy
- reduceByKey
- foldByKey
- combineByKey
- groupByKey
- cogroup
- join, leftOuterJoin, ..
- sortByKey
- lookup



- cache to set one/best partitioning scheme
- filter early
- spanByKey()
- spanBy(f)
- repartitionByCassandraReplica(...)
- applyPartitionerFrom(...)

```
case class My_Record (pk: Int, value: String)
```

```
//
```

```
val rows1 = sc.parallelize(Seq(  
  new My_Record(400, "K-Mart"),  
  new My_Record(401, "Wal*Mart"),  
  new My_Record(402, "Piggly Wiggly")  
))
```

```
println(rows1.partitions.size)
```

```
// 3
```

```
println(rows1.partitioner)
```

```
// None
```

```
// This box has 4 cores, DSE cores defaults to 4 - 1
```

```
val rows2 = rows1.repartition(2*sc.defaultParallelism)
```

```
println(rows2.partitions.size)
```

```
// 6
```

Partitioning: Examples

Partitioning: Examples

```
// To change partitioner, need a K/V pair RDD  
// Since pk is unique, just need a groupBy
```

```
val rows2_kv = rows2.groupBy(_.pk)
```

```
rows2_kv.collect().foreach(println)
```

```
(402,CompactBuffer(My_Record(402,Piggly Wiggly)))  
(400,CompactBuffer(My_Record(400,K-Mart)))  
(401,CompactBuffer(My_Record(401,Wal*Mart)))
```

```
println(rows2_kv.partitioner)  
// Some(org.apache.spark.HashPartitioner@6)
```



Why 6 ?

Partitioning: Examples

```
val rows3 = rows2_kv.partitionBy(new org.apache.spark.HashPartitioner(9))  
println(rows3.partitioner)  
// Some(org.apache.spark.HashPartitioner@9)
```

Partitioning Matching DSE: Examples

```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state")
```

```
println(c_rows.partitioner)  
// None
```

```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state").  
  keyBy[Tuple1[Int]]("customer_num", "company", "state")
```

```
println(c_rows.partitioner)  
// Some(com.datastax.spark.connector.rdd.partitioner.CassandraPartitioner@4da90f6c)
```



Which is better ?

**** First Url on Notes page**

Partitioning Matching DSE: Examples, RDD

```
case class My_Record (customer_num: Int, company: String, state: String)
//
val rows1 = sc.parallelize(Seq(
  new My_Record(400, "K-Mart", "MI"),
  new My_Record(401, "Wal*Mart", "AR"),
  new My_Record(402, "Piggly Wiggly", "WI")
))
```

Partitioning Matching DSE: Examples, RDD

```
val rows2 = rows1.repartitionByCassandraReplica("ks_7571", "customer", 10)

println(rows2.partitioner)
// Some(com.datastax.spark.connector.rdd.partitioner.ReplicaPartitioner@4b3be6f7)
println(rows2.partitions.size)
// 10
```



Use case ?

Partitioning Matching DSE: Examples, DSE Table

```
val o_rows1 = sc.cassandraTable("ks_7571", "orders").  
  select("order_num", "customer_num", "order_date").  
  keyBy[Tuple1[Int]]("order_num", "customer_num", "order_date")
```

```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company", "state").  
  keyBy[Tuple1[Int]]("customer_num", "company", "state")
```

```
val o_rows1b = o_rows1.applyPartitionerFrom(c_rows)
```

Writing to DSE:



Same Action:

- RDD of tuples
- RDD of case class objects
- RDD of CassandraRow objects

Writing to DSE:

API Call

- `saveToCassandra(ks, tb [, SomeColumns(c1, c2, ..)])`
insert/update (upserts all), delete
- `saveAsCassandraTable(ks, tb [, SomeColumns(c1, c2, ..)])`
Makes table too, first column PK only
- `saveAsCassandraTableEx(tab-def [, SomeColumns(c1, c2, ..)])`
Makes table *extended*, more property control

Mechanics of this-

- CQL insert for every row in the RDD, grouped in unlogged batches
- Default CL is LOCAL_QUORUM

Writing to DSE: Tuples

```
val rows1 = sc.parallelize(List(
  (400, "K-Mart", "MI"),
  (401, "Wal*Mart", "AR"),
  (402, "Piggly Wiggly", "WI")
))
rows1.saveToCassandra("ks_7571", "customer",
  SomeColumns("customer_num", "company", "state"))
```

```
val rows2 = sc.parallelize(List(
  (500, "New Company", "AK")
))
rows2.saveToCassandra("ks_7571", "customer",
  SomeColumns("customer_num" as "_1", "company" as "_3", "state" as "_2"))
```



Controlling
column
assignments

Writing to DSE: case class objects

```
case class My_Record (customer_num: Int, company: String, state: String)
//
val rows3 = sc.parallelize(Seq(
  new My_Record(500, "BlockBuster", "TX"),
  new My_Record(501, "NetFlix", "CA"),
  new My_Record(502, "RST Video", "NJ")
))

rows3.saveToCassandra("ks_7571", "customer",
  SomeColumns("customer_num", "company", "state"))
```

Writing to DSE: CassandraRow Objects

```
val rdd1 = sc.cassandraTable("ks_7571", "customer").  
  where("customer_num = 101").  
  select("customer_num", "company", "state")
```

```
val rdd2 = rdd1.map{ r => (701, "New Company", "AK") }  
// rdd2.collect().foreach(println)
```

```
rdd2.saveToCassandra("ks_7571", "customer",  
  SomeColumns("customer_num", "company", "state"))
```

Write to DSE: if not exists

```
case class My_Customer (customer_num: Int,  
    company: String, state: String)  
  
val df1 = spark.createDataset(Seq(My_Customer(901,  
    "New Company 1", "AK")))  
val df2 = spark.createDataset(Seq(My_Customer(901,  
    "New Company 2", "AK")))  
  
df1.write.format("org.apache.spark.sql.cassandra").  
    options(Map("table" -> "customer", "keyspace" ->  
        "ks_7571", "spark.cassandra.output.ifNotExists" ->  
            "true")).mode(SaveMode.Append).save()
```



write() is
actually a
DataFrame
method, not
RDD

Write to DSE: if not exists

```
cqlsh> select * from customer;
```

customer_num	address1	address2	city	company	fname	lname	phone	state	zipcode
901	null	null	null	New Company 1	null	null	null	AK	null

```
df2.write.format("org.apache.spark.sql.cassandra").  
  options(Map("table" -> "customer", "keyspace" -> "ks_7571",  
    "spark.cassandra.output.ifNotExists" ->  
      "true")).mode(SaveMode.Append).save()
```

```
// same output as first line; write did not occur
```

Writing to DSE: (collections, UDTs, TTL, nulls)

It's all there; nulls in particular is above Scala-101

Supports operations on collections:

- append/add, (lists, sets, maps)
- prepend, (lists)
- remove, (lists, sets)
- overwrite, (lists, sets, maps) (Default)
- UDTs, case class, and `UDTValue.fromMap( ... )`
- Tombstones; needs advanced Scala function
- TTL/WRITETIME, uses `WriteConf ()` (DataFrame for now ?)

See Url on Notes page for examples to all-

Writing to DSE: Deletes

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  filter(r => r.getInt("customer_num") > 102)  
rows.collect().foreach(println)  
CassandraRow{customer_num: 103, ...  
CassandraRow{customer_num: 104, ...
```

```
rows.deleteFromCassandra("ks_7571", "customer")
```

```
val rows = sc.cassandraTable("ks_7571", "customer")  
rows.collect().foreach(println)  
CassandraRow{customer_num: 102, ...  
CassandraRow{customer_num: 101, ...
```

Writing to DSE: Deletes, specific columns

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  filter(r => r.getInt("customer_num") > 102)
```

```
rows.deleteFromCassandra("ks_7571", "customer",  
  SomeColumns("state", "city"))
```

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "city", "state")  
rows.collect().foreach(println)
```

```
CassandraRow{customer_num: 103, city: null, state: null}
```

```
CassandraRow{customer_num: 104, city: null, state: null}
```

```
CassandraRow{customer_num: 102, city: San Francisco, state: CA}
```

```
CassandraRow{customer_num: 101, city: Sunnyvale, state: CA}
```

Writing to DSE: Deletes, partition deletes

- DSE Core has an optimization to delete an entire partition, called a "range delete"
- Supported here too
- Can also delete on a time stamp to row
- Check the Jira on the Notes page.



End of Unit:



Additional Detail:

Shared Variables:

- Not specific to RDDs
- Actually, hard to do on a shared nothing platform
- Broadcast variables
 - Read-only at Task level
 - Writable at Driver level
 - Workers move this around
 - Use case ? Symbol table, other
- Accumulators
 - Native support for numeric (double, long, float), can program other types
 - Use case ? Counting, aggregation
 - Returned to driver for final (sum)



```
val bv = sc.broadcast(Set(102, 104))
```

```
val ac = sc.accumulator(0)
```

```
val rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "company").  
  filter(r => bv.value contains r.getInt("customer_num")).  
  cache
```

```
rows.collect().foreach(println)  
CassandraRow{customer_num: 104, company: Play Ball!}  
CassandraRow{customer_num: 102, company: Sports Spot}
```

```
rows.foreach{ r => ac += r.getString("company").length }  
println(ac)  
31           // Will add across executors, nodes
```

Shared Variables: Example



Why the cache ?